Project document

Project Title

Tarot reading app

Project Overview

I plan to develop a tarot app allowing users to pick cards and read tarot. This mobile app will simulate shuffling a tarot card deck, enabling the user to select cards from the 22 Major Arcana and provide interpretations of their meanings. The purpose of this application is to provide enjoyment and a good interactive experience for users, and the target audience is tarot enthusiasts, spiritual seekers, and users interested in divination entertainment. The app consists of four pages: the home page, the shuffle and select card page, the card flip page, card reading page. The key features are realistic card simulation, shuffling, selection and flip, complete database for 22 major Arcana cards and smooth user experience, clear layout and intuitive navigation.

Learning Plan

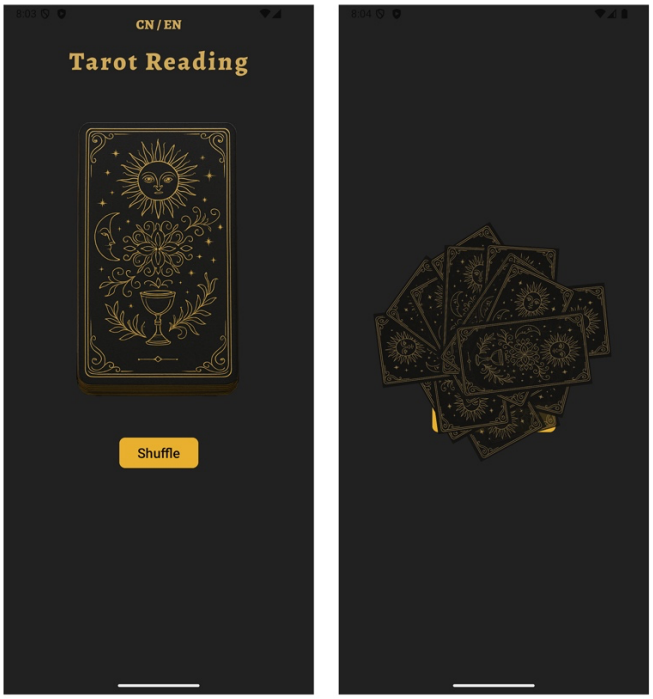
While completing this tarot card reading app, I plan to reinforce and deepen my understanding of key Android development concepts taught in previous practices. In earlier assignments, I learned to develop a range of small apps, including a loan calculator, a drawing app, a mini-game, and a recipe app. These exercises introduced important components such as Fragments, Room database, DAO, string resources, and user interaction logic. By working on this project, I aim to consolidate my knowledge in the following aspects: fragments and navigation, learn how to effectively manage multiple screens by using fragments and navigation components. How to manage massive data and how to transfer data, and try to achieve some simple animations to enhance user experience and rich my functions.

This project is a practice of my previous lessons and an opportunity to apply them in a more complete and real-world application. It also helps me practice designing an app with both function and user experience in mind.

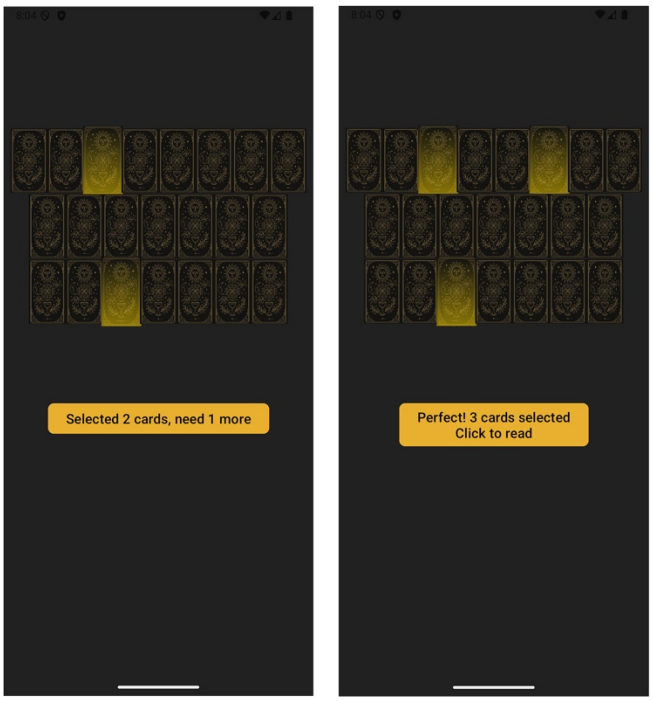
Time usage

Date	Description	Total Hours
15/04/2025	Build a project file and git push to GitHub	0.5 hours
16/04/2025	Write project brief	1 hours
10/05/2025	Decided final project content and wrote a brief proposal	3 hours
18/05/2025	Research similar online projects, collect free tarot card images, and create the cover image	3.5 hours
20/05/2025	Build 22 tarot card database and related java classes	4 hours
21/05/2025	Build the main page layout and create style.xml	3 hours
22/05/2025	Build a shuffle card page and create a shuffle effect animation	8.5 hours
23/05/2025	Improve the shuffle effect and card selection features.	4 hours
24/05/2025	Build card flip page and card reading page, use navigation and card passing logic by NavHost	4 hours
25/05/2025	Improve card reading layout, including card image and its descriptions.	2 hours
26/05/2025	Write Project document	3 hours
27/05/2025	Add language option, test app, add comment and tried to generate Javadoc	4 hours
28/05/2025	Generate a class UML diagram, error with auto-generate Javadoc, replaced by adding an index.html	3 hours
29/05/2025	Write Project document	3 hours

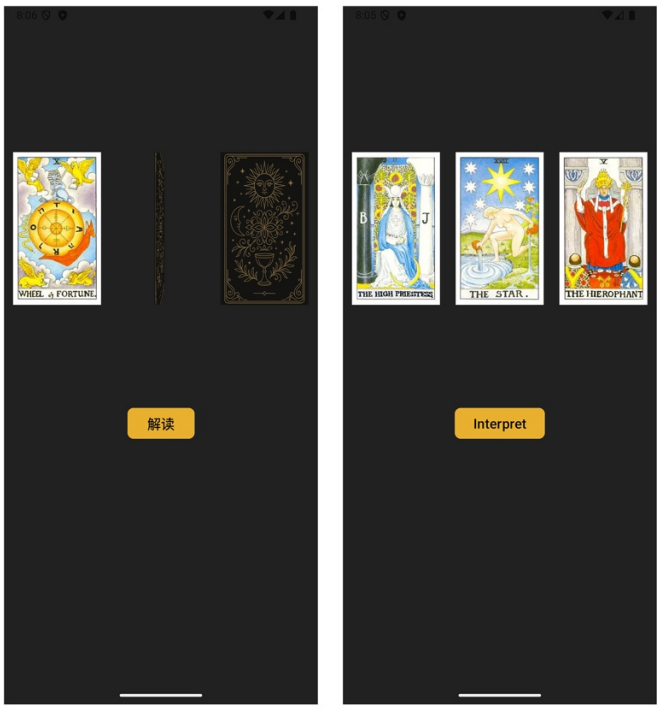
Screenshots of the user interface



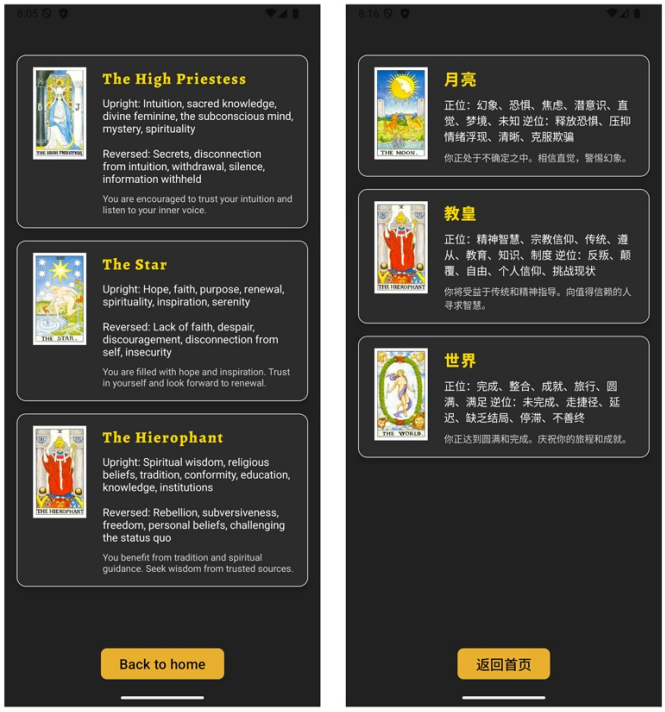
Step 1: Select preferred language and click the shuffle button



Step 2: After the card shuffle, select 3 tarot cards and click to next

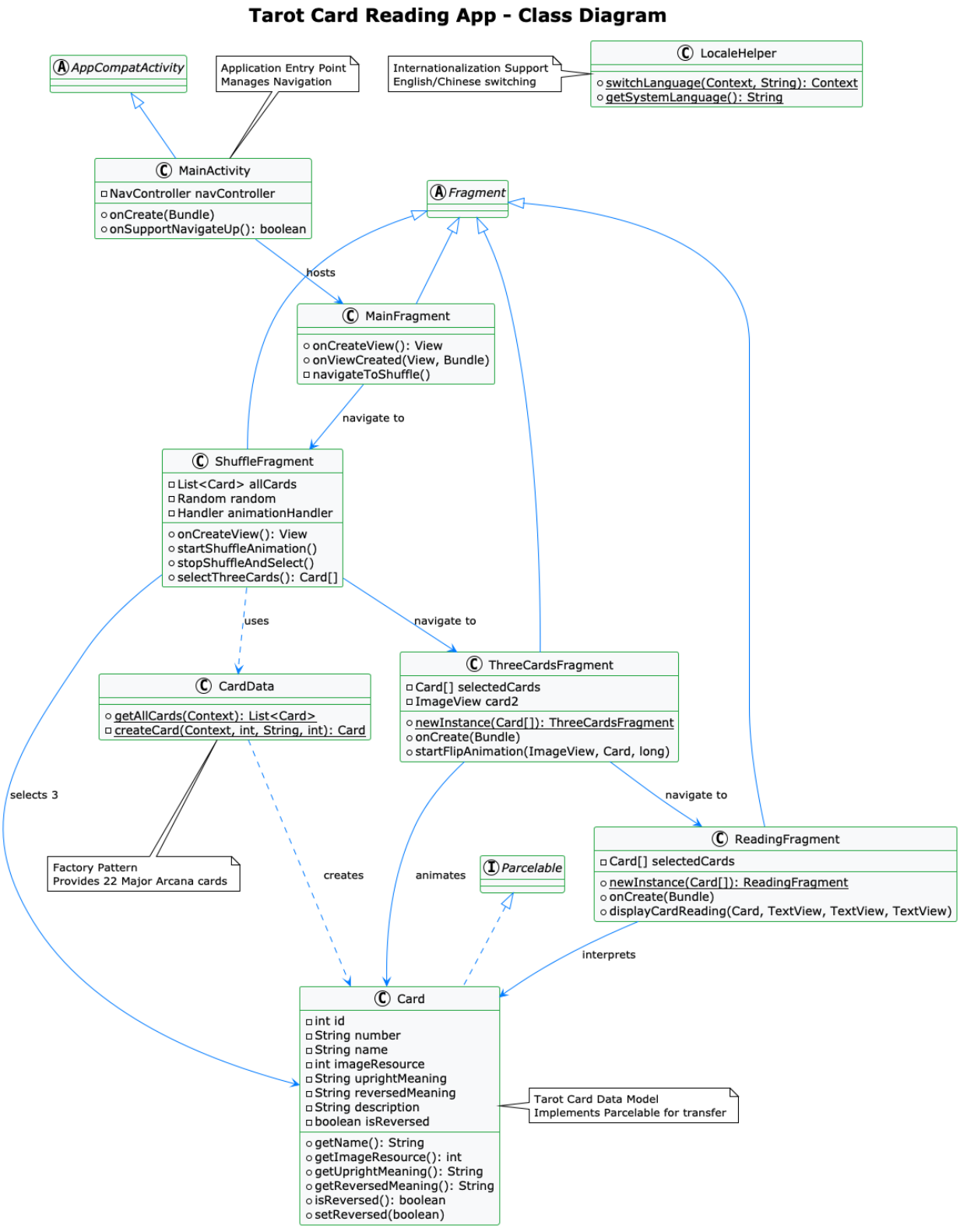


Step 3: The Selected card will flip and display, click to inteprete



Step 4: Get the explanation of the card, and click to return home page

A visualization and textual description of all program components used in the project



This App is structured around a main activity that manages navigation between several fragments, each responsible for a specific part of the tarot reading flow (main screen → shuffle → select cards → reading). Data is modelled with a Card class, and all card data is provided by a factory (CardData)—language switch is handled via a helper class and string resources.

MainActivity

Role: The entry point of the app, responsible for managing navigation between fragments.

Key Members:

NavController navController

Responsibilities:

Hosts the main navigation logic.

Supports switching between English/Chinese

Fragments

MainFragment

Role: The main/home screen fragment.

Responsibilities:

Displays the deck image and shuffle button.

Navigates to the ShuffleFragment when the user wants to shuffle.

ShuffleFragment

Role: Handles the shuffle animation and card selection.

Key Members:

List<Card> allCards (all tarot cards)

Random random

Handler animationHandler

Responsibilities:

Animates the shuffling of cards.

Randomly selects three cards when shuffling stops.

Navigates to ThreeCardsFragment with the selected cards.

ThreeCardsFragment

Role: Displays the three selected cards.

Key Members:

Card[ ] selectedCards

ImageView card2

Responsibilities:

Shows the selected cards.

Handles card flip animation.

Navigates to ReadingFragment for card interpretation.

ReadingFragment

Role: Shows the interpretation/reading for the selected cards.

Key Members:

Card[ ] selectedCards

Responsibilities:

Displays the meanings and descriptions for each card.

CardData

Role: Factory/provider for tarot card data.

Responsibilities:

Provides a list of all 22 Major Arcana cards.

Uses string resources for switching languages.

Creates Card objects with all necessary info.

Card

Role: Data model for a tarot card.

Fields:

int id

String number

String name

int imageResource

String uprightMeaning

String reversedMeaning

String description

boolean isReversed

Implements: Parcelable (for passing between fragments)

Responsibilities:

Stores all card data.

Provides getters/setters for card properties.

Language option

LocaleHelper

Role: Handles language switching and locale management.

Responsibilities:

Switches the app language between English and Chinese.

Retrieves the current system language.

Relationships Between Components

MainActivity hosts and manages navigation between all fragments.

MainFragment is the entry fragment, which navigates to ShuffleFragment.

ShuffleFragment uses CardData to get all cards, animates shuffling, and selects three cards, then navigates to ThreeCardsFragment.

ThreeCardsFragment receives the selected cards, displays them, and can navigate to ReadingFragment for interpretation.

ReadingFragment receives the selected cards and displays their meanings.

Card objects are passed between fragments using the Parcelable interface.

LocaleHelper is used by the app to support language switching.

Self-Evaluation

The project achieves the basic function of my plan, simple and usable. And the brief written in the preparation stage gives me a clear guide to achieve all elements and complete them. The core functionality outlined in the design document, including card shuffling simulation, interactive card selection from the 22 Major Arcana, and interpretations, was fully realized in the final application. There are many challenges in the process, but I learnt a lot after solving them. I learned how to handle Intent-based navigation and passing data between different pages. Working with Room Database makes me know how to effectively store and retrieve tarot card information locally. Developing the card-shuffling animations and visual effects helped me understand Android's animation frameworks. In addition, I realized the importance of regular Git commits and pushes. When the application encountered an unexplained runtime issue, being able to revert to a previous working version saved my time and prevented data loss. Each problem I overcame contributed to a deeper understanding of Android Studio's principles and strengthened my ability to debug and troubleshoot complex issues.

Challenges and Ideas for the Future

The development process has some challenges. The card-shuffling animation is the most time-consuming challenge, I tried many times to achieve a realistic effect that simulates physical cards shuffle naturally, adjusting data and testing. Then, making the card display similar to a hand-arranged stack together was particularly complex and required multiple tests to achieve the desired visual effect. In addition, how to make the selected card naturally float is facing some problems, each time it will fly out and leave a big distance from the rest cards, this constant issue force me to find another way to show the effect, so I using slightly floating and scaling effects with gradients to indicate selection. Generating JavaDoc is another issue; it keeps failing and reports errors due to Gradle dependency conflicts. Despite trying multiple online solutions, which still do not work, I have to create a simplified manual version.

The current application only uses 22 Major Arcana, which is limited for the user to select. Adding the complete 78-card tarot deck with all Minor Arcana suits will provide more comprehensive reading experiences. And so far, my app only allows users to select three cards, but in real tarot, there are many options, Celtic Cross, Past-Present-Future layouts, and specialized spreads for relationships, career, and personal growth. There are some possible functions to improve, the tarot layout for different purposes, reading history storage, and a summary of monthly or yearly records. and enhance animations and layout.

Course Feedback

In my opinion, the course structure is well-designed, and the four exercises range from easy to complex, which allows me to gradually build an understanding of Android programming concepts, starting from fundamentals and advancing to more complex implementations. Each exercise built upon the previous one, creating a solid foundation of knowledge that prepared me for the final project. The screenshots of the course materials were helpful in transcending language barriers. If it provides an English version will be better, because some translations might misunderstand the content, and the last exercise, even if I follow all the steps in the PPT, some gaps make my program not work, so I checked the source code and corrected them. Overall, it's a nice learning experience.